

Releasing a new version of Wren

A step-by-step checklist: deploy the site, update the repository, publish the release.

Prepared for Alan Paynter · wrencms.com

Before you start

You should have four things from the chat: **index.php**, **CHANGELOG.md**, sometimes **README.md**, and the release **zip**. The first three go into the repository. The zip only ever goes on the GitHub release page.

The golden rule. Unzip downloads in your **Downloads** folder, never inside **wren-cms**. Copy only the loose files into the repository. If a folder or a .zip ever lands inside wren-cms, it will end up committed — that is what caused every tangle so far.

The two folders, and never confusing them

Code repository	~/Documents/wren-cms branch: main
Wiki repository	~/Documents/wren-cms/wren-cms.wiki branch: master

Whenever a command misbehaves, the first question is always “which folder am I in?” Type `pwd` to find out.

Part 1 — Update the live site first

Always deploy before publishing, so the site is never behind the release it advertises.

1 Upload index.php

Upload the new **index.php** over the old one in the web root of wrencms.com, using your host’s file manager or FTP. Nothing else needs uploading: your content, settings, images and theme are untouched, and the database updates its own structure on the next page load.

2 Check it came up

Load the site, then sign in to the admin and confirm the new feature is present. A hard refresh (Cmd+Shift+R) helps if the change involves styling.

Part 2 — Update the repository

3 Put the files in place

Unzip the download in **Downloads**. Copy **index.php**, **CHANGELOG.md** and (when it changed) **README.md** into `~/Documents/wren-cms`, replacing the existing files. Do not copy folders. Do not copy the zip.

4 Open Terminal in the repository

Type `cd` followed by a space, drag the **wren-cms** folder onto the Terminal window, and press Enter. Then confirm where you are:

```
pwd
```

It must print exactly `/Users/alanpaynter/Documents/wren-cms` — no extra folders on the end.

5 Collect any changes made on GitHub

If you have edited the README or anything else in the browser, this brings it down so the two ends agree:

```
git pull origin main --no-edit
```

If it reports a **conflict**, stop. Open the named file, delete the `<<<<<<<`, `=====` and `>>>>>>>` marker lines, keep the text you want from both sides, save, then run `git add .` and `git commit --no-edit` before carrying on.

6 Check what git can see

```
git status
```

You want **only** the files you copied, listed as modified. If any folder or `.zip` appears, delete it before going further — for example:

```
rm -rf "Wren CMS 1.6.0 Stable"
rm -f *.zip
```

7 Commit and push

Run these one line at a time, reading the result of each before typing the next:

```
git add -A
git commit -m "Wren 1.6.0 — one canonical address per page"
git push
```

Replace the message with the version and summary for the release you are shipping. A successful push ends with a line like `7f152ab..aaddca3 main -> main`.

8 Tag the version

```
git tag -a v1.6.0 -m "Wren 1.6.0 'Hypoleucus'"
git push origin v1.6.0
```

Tags are what the release page attaches to. Use the real version number and the release's wren-species name.

Part 3 — Publish the release on GitHub

9 Draft the release

Go to github.com/Wren-CMS/wren-cms → **Releases** → **Draft a new release**.

- **Choose a tag:** pick the tag you just pushed from the dropdown — do not type a new one.
- **Title:** the version and its name, e.g. Wren 1.6.0 “Hypoleucus”.
- **Notes:** what changed, in a sentence or two, ending with “Upgrading: replace index.php.”
- **Attach:** drag in the release zip.
- **Release label:** choose **None**. This means a normal stable release; the newest one automatically gets the green “Latest” badge.

10 Verify

The Releases page should show your new version carrying the green **Latest** badge. That badge is what the Download button on wrencms.com and the `/download` short link resolve to, so nothing else needs editing.

Optional — update the wiki

If the release changed how something works, the wiki should say so. The wiki is a separate repository on the **master** branch:

```
cd ~/Documents/wren-cms/wren-cms.wiki
git pull
# copy the updated .md files in, then:
git add -A
git commit -m "Document the new feature"
git push
```

If something goes wrong

What you see	What to do
! [rejected] ... fetch first	GitHub has a change you do not. Run <code>git pull origin main --no-edit</code> , then push again.
Need to specify how to reconcile divergent branches	Run <code>git config --global pull.rebase false</code> once, then pull again. It never asks again.
CONFLICT (content): ...	Edit the file, remove the marker lines, keep what you want, then <code>git add .</code> and <code>git commit --no-edit</code> .
not a git repository	You are in the wrong folder — probably an unzipped release. <code>cd ~/Documents/wren-cms</code> and check with <code>pwd</code> .
Commit lists folders you did not expect	Undo it with <code>git reset --soft HEAD~1</code> , delete the stray folders and zips, then commit again.
Committing is not possible ... unmerged files	A conflict is still open. Fix the marked file, <code>git add</code> it, then commit.

Nothing you do locally can damage what is on GitHub. Commits, resets and conflicts all happen on your Mac; only `git push` sends anything. If the terminal looks a mess, stop and read the last message rather than typing another command — and never run five commands as one paste.

The whole thing, on one line each

Once it is familiar, this is the entire procedure:

```
# upload index.php to the site first, then:
cd ~/Documents/wren-cms
git pull origin main --no-edit
# copy in index.php, CHANGELOG.md, README.md
git status
git add -A
git commit -m "Wren X.Y.Z — summary"
git push
git tag -a vX.Y.Z -m "Wren X.Y.Z 'Name'"
git push origin vX.Y.Z
# then draft the release on GitHub and attach the zip
```